WhatsBook

WhatsApp-based Venue Booking & Payments

MVP Architecture, Flowcharts, Database Design and Implementation Blueprint

The product in one sentence

Venue owners configure their venue, rates and availability in a web dashboard. Customers message a venue-specific WhatsApp link, open a booking page, choose a slot and pay by UPI. WhatsBook tracks the booking and sends the right confirmation.

Scope: a focused first release for independent venue owners. This document intentionally excludes marketplace features, advanced analytics, loyalty, promotions and complex automation.

Prepared as a practical starting point for product planning and software development

1. Product overview

WhatsBook is a lightweight SaaS booking system for venues. Each venue owner registers and subscribes, configures venue details and pricing, and receives a unique venue code plus WhatsApp entry link. Customers use that WhatsApp entry point to reach the venue's booking page without installing another app.

Area	MVP responsibility
Venue owner web app	Register/login, subscription status, venue profile, pricing, operating hours, slot blocking and booking review.
WhatsApp	Receive the venue code and send the customer a secure booking-page link; send booking status messages.
Customer booking page	Show venue details, date, available slots, price and UPI payment instructions.
Booking backend	Validate availability, hold slots temporarily, create bookings, record payment status and prevent double booking.
Database	Store owners, subscriptions, venues, pricing, availability blocks, bookings and payment records.

Core product principle

WhatsApp is the communication channel. The owner web app is the control centre. The backend and database are the source of truth for slot availability, bookings and payment state.

2. User roles and interfaces

User / interface	What they can do
Venue owner	Create account, subscribe, set up venue, set hourly/half-hourly prices, define operating hours, block slots for offline bookings, review pending bookings and approve/reject where required.
Customer	Start on WhatsApp, open the booking link, select date and slot, view price, pay using UPI and receive booking status on WhatsApp.
WhatsBook administrator (minimal internal role)	Support owner accounts and resolve operational issues. An elaborate admin console is not required for the first pilot.

3. High-level system architecture

Build one backend application with clearly separated modules—not a set of microservices. This keeps hosting, deployment, debugging and development cost down while leaving room to split components later if the product grows.

```
flowchart TB
VO[Venue Owner] --> WEB[Owner Web App]
CU[Customer] --> WA[WhatsApp Business API]
WA -->|Venue code / message| API[WhatsBook Backend API]
WA -->|Booking link| BOOK[Customer Booking Page]
WEB --> API
BOOK --> API

API --> AUTH[Registration & Subscription]
API --> VENUE[Venue & Pricing]
API --> SLOT[Availability & Slot Engine]
API --> BM[Booking Manager]
API --> PM[Payment Status Manager]
API --> NM[Notification Manager]

AUTH --> DB[(PostgreSQL)]
VENUE --> DB
SLOT --> DB
BM --> DB
PM --> DB
```

```
BOOK --> UPI[UPI App / QR]
UPI -. payment verification .-> PSP[Payment Provider / Webhook]
PSP -. verified result .-> PM
NM --> WA
NM --> WEB
```

Architecture components

Component	Purpose
Owner web app	Account access, venue configuration, price setup, calendar/slot blocking and booking dashboard.
Customer booking page	Mobile-friendly page opened from WhatsApp; displays live availability and UPI instructions.
WhatsApp integration	Receives inbound messages, validates venue code and sends approved message templates/booking links.
Backend API	Applies business rules and coordinates all application modules.
Slot engine	Calculates available time ranges, creates temporary holds and prevents overlapping confirmed bookings.
Payment manager	Stores payment attempts and provider references; supports manual owner verification if automatic confirmation is unavailable.
Notification manager	Sends customer WhatsApp updates and updates the owner dashboard.
PostgreSQL	Persists all business records and enforces data consistency.

4. Customer booking flow

Happy path and exception path

```
flowchart TD
A[Customer opens venue WhatsApp link] --> B[Customer sends venue code]
B --> C[WhatsApp webhook reaches backend]
C --> D{Code valid and active?}
D -->|No| E[Send helpful error message]
D -->|Yes| F[Send customer booking URL]
F --> G[Customer opens booking page]
G --> H[Select date]
H --> I[Backend returns available slots]
I --> J[Customer selects slot]
J --> K{Slot still available?}
K -->|No| I
K -->|Yes| L[Create temporary slot hold]
L --> M[Display price and UPI payment instructions]
M --> N[Customer attempts UPI payment]
N --> O{Payment automatically verified?}
O -->|Yes| P[Mark payment paid and booking confirmed]
O -->|No| Q[Mark awaiting owner approval]
Q --> R[Owner checks UPI transaction in dashboard]
R --> S{Owner approves?}
S -->|Yes| P
S -->|No| T[Reject booking and release slot]
P --> U[Send WhatsApp confirmation]
P --> V[Show confirmed booking in owner dashboard]
T --> W[Notify customer of rejection]
E --> X[End]
```

Critical rule: A click on a UPI button, a QR-code display, or a customer statement is not proof of payment. Only a trusted payment-provider verification event or an owner's explicit manual verification can mark payment as paid. Send a confirmed-booking message only after the booking reaches the confirmed state.

5. Venue owner flow

```
flowchart TD
A[Owner visits WhatsBook] --> B[Register and verify account]
```

```
B --> C[Choose subscription]
C --> D[Subscription active]
D --> E[Create venue profile]
E --> F[Generate unique venue code and WhatsApp link]
F --> G[Set operating hours and pricing]
G --> H[Owner dashboard active]
H --> I{What happens next?}
I -->|Customer booking| J[Review booking and payment status]
J --> K{Payment verified / approval needed?}
K -->|Verified| L[Confirm booking]
K -->|Needs manual check| M[Check UPI transaction]
M --> N{Approve?}
N -->|Yes| L
N -->|No| O[Reject and release slot]
I -->|Offline phone booking| P[Select date and time in calendar]
P --> Q[Block slot]
Q --> R[Availability updated immediately]
L --> S[Dashboard and customer WhatsApp updated]
O --> S
```

6. Slot and availability rules

Slot management is the most important reliability feature. The system must treat a venue's time as a set of bookable intervals and must never allow overlapping active holds or confirmed bookings for the same venue.

Online booking

• Customer chooses a date and a start/end time or a configured duration.

• Backend checks operating hours, existing manual blocks, active holds and confirmed bookings.

• If available, backend creates a temporary hold (for example, 10 minutes) and returns an expiry time.

• Customer proceeds to UPI. The hold prevents another customer from taking the same slot while payment is being completed.

• On verified payment or owner approval, convert the hold into a confirmed booking. On expiry, failure or rejection, release it.

Manual/offline booking

• Owner opens the calendar, chooses date and time, enters a short note/reference if useful, and selects Block Slot.

• Backend checks for conflicts before saving the block.

• Blocked time is immediately excluded from customer availability.

• Owner can unblock a slot only if it has not been converted into a confirmed booking or otherwise restricted.

Concurrency requirement

Availability check and slot-hold creation must be protected by a database transaction/constraint. Two customers may request the same slot at nearly the same time; only one may succeed. Do not rely only on a frontend availability check.

7. Booking and payment states

Keep booking state separate from payment state. They describe different things and should not be overloaded into one status field.

Booking status	Meaning
PENDING_PAYMENT	Slot held while customer attempts payment; hold has an expiry.
AWAITING_OWNER_APPROVAL	Payment cannot be automatically verified; owner must check and decide.

Booking status	Meaning
CONFIRMED	Venue and time are reserved. Customer confirmation may be sent.
REJECTED	Owner rejected the request; release the slot if no longer reserved.
EXPIRED	Payment/hold window elapsed without successful completion.

Payment status	Meaning
PENDING	No trusted confirmation received yet.
PAID	Payment provider verified the transaction, or owner explicitly verified it.
FAILED	Provider reports failure or payment attempt failed.
MANUALLY_VERIFIED	Optional audit label if you want to distinguish owner-checked UPI from provider-confirmed payment.

Recommended decision logic

• **Automatic verification available:** verified payment → mark payment PAID → confirm booking → notify customer and owner.

• **Automatic verification unavailable:** keep booking AWAITING_OWNER_APPROVAL → notify owner → owner checks bank/UPI receipt and approves or rejects → notify customer.

• **Do not silently assume payment:** a screenshot or typed UPI reference is evidence for the owner to inspect, not automatic proof.

8. MVP database schema

Start with these core tables. Use UUID or another non-guessable primary key. Include created_at and updated_at timestamps on operational records. Store money as integer paise (or an exact decimal type), never floating point.

Table	Suggested fields
owners	id, name, email, phone, password_hash, account_status, created_at
subscriptions	id, owner_id, plan_code, status, starts_at, expires_at, provider_reference
venues	id, owner_id, venue_name, unique_code, address, timezone, operating_hours, active_status
pricing	id, venue_id, duration_minutes, price_paise, effective_from, active_status
availability_blocks	id, venue_id, starts_at, ends_at, block_type, note, created_by
bookings	id, venue_id, customer_name (optional), customer_phone, starts_at, ends_at, amount_paise, booking_status, hold_expires_at, source
payments	id, booking_id, amount_paise, payment_status, provider, provider_reference, upi_reference, verified_by, verified_at

Data integrity rules

• A venue unique_code must be unique and difficult to guess; it is an identifier, not an authentication secret.

• A booking must belong to one venue and one customer contact; payment records must reference a booking.

• Store venue timezone and interpret all booking times consistently. For an India-first MVP, use Asia/Kolkata as the default while keeping timezone configurable.

• Prevent overlapping confirmed bookings, active holds and owner-created blocks for the same venue. Define explicitly whether a block can overlap a pending hold and how conflicts are resolved.

• Keep an audit trail for owner approval/rejection and manual payment verification: who acted, when, and what changed.

9. Suggested API endpoints

Illustrative REST endpoints. Names can be adjusted to the chosen framework; the responsibilities are the important part.

Area	Method & endpoint	Purpose
Owner auth	POST /api/auth/register	Create owner account.
Owner auth	POST /api/auth/login	Start authenticated owner session.
Subscription	POST /api/subscriptions	Start/renew subscription or record plan activation.
Venue	POST /api/venues	Create venue and unique code.
Venue	GET /api/venues/:id	Read venue settings.
Venue	PATCH /api/venues/:id	Update venue details and operating hours.
Pricing	PUT /api/venues/:id/pricing	Create/update hourly and half-hourly rates.
WhatsApp	POST /api/webhooks/whatsapp	Receive and validate WhatsApp webhook events.
Availability	GET /api/public/venues/:code/availability	Return available slots for date/duration.
Booking	POST /api/public/bookings/holds	Check and temporarily hold a slot.
Booking	POST /api/public/bookings/:id/checkout	Create payment instructions or payment session.
Payment	POST /api/webhooks/payments	Receive trusted provider payment updates.
Owner bookings	GET /api/owner/bookings	List owner bookings and their statuses.
Owner action	POST /api/owner/bookings/:id/approve	Manually approve after payment check.
Owner action	POST /api/owner/bookings/:id/reject	Reject and release slot where valid.
Calendar	POST /api/owner/availability-blocks	Block a slot for offline/manual booking.
Calendar	DELETE /api/owner/availability-blocks/:id	Remove an eligible manual block.

10. WhatsApp interaction

Example customer journey

Customer: sends the venue code to the venue's WhatsApp entry point.

WhatsBook: validates the code and replies with the venue name and a secure booking URL.

Customer: opens the mobile booking page, chooses date/time and pays by UPI.

WhatsBook: sends a booking-confirmed or action-required message only when the corresponding state is reached.

Implementation notes

• Use the official WhatsApp Business Platform/Cloud API and configure webhook verification and message delivery handling.

• Use approved message templates where WhatsApp policy requires them, particularly for business-initiated messages outside the customer-service window.

• Store WhatsApp message IDs and delivery/error status so failed notifications can be retried safely.

• Do not make WhatsApp the booking database. A customer should still be able to open the booking link if a message is delayed; the backend remains authoritative.

11. UPI and payment design

UPI is the payment method customers use, but the app still needs a reliable way to learn whether money arrived. A static QR code or UPI intent does not, by itself, provide trustworthy server-side payment confirmation.

Approach	How it works	MVP implication
Payment provider with UPI support	Create a payment/order; provider sends a signed webhook or server-verifiable status.	Preferred for automatic confirmation. Verify webhook signature and transaction/order reference.
Manual UPI verification	Show UPI details/QR; owner checks their bank or UPI app and approves in dashboard.	Simpler fallback. Booking remains awaiting owner approval until the owner acts.

Never trust a browser redirect alone as proof of payment. Verify the provider's server-side status or signed webhook. Make webhook processing idempotent so duplicate events do not create duplicate confirmations.

12. Recommended technology stack

Layer	Practical option	Why
Frontend	Next.js / React	One codebase can serve owner dashboard and public booking page.
Backend	Node.js API (e.g. Next.js route handlers or a separate Express/NestJS app)	Use a modular monolith for the MVP.
Database	PostgreSQL	Transactions, constraints and relational booking data.
Hosting	Managed app hosting or a small cloud VPS	Choose based on developer experience and expected traffic.
WhatsApp	Meta WhatsApp Business Cloud API	Official business messaging integration.
Payments	UPI-capable payment provider with webhooks; manual UPI fallback	Supports automatic verification where available.
Authentication	Secure server-side sessions and password hashing	Protect owner dashboard and actions.

13. MVP feature boundary

Build in MVP	Defer until later
Owner registration/login	Marketplace and venue discovery
Subscription activation/status	Advanced subscription billing and coupons
Unique venue code and WhatsApp link	Multiple branches and complex staff roles
Venue profile and operating hours	Custom themes and extensive branding
Hourly/half-hourly rates	Dynamic pricing and seasonal rules

Build in MVP	Defer until later
Date/slot availability	Recurring bookings and complex packages
UPI payment and manual verification fallback	Refund automation and split payouts
Owner approve/reject and manual slot block	Advanced analytics, loyalty and promotions
WhatsApp status messages and owner dashboard	Reminders, campaigns and marketing automation

14. Non-functional requirements and safeguards

• **Security:** hash passwords, use HTTPS, protect owner APIs with authentication/authorization, validate inputs and rate-limit public endpoints.

• **Tenant isolation:** every owner-facing query must be scoped to the authenticated owner's venue. Never trust a venue_id sent by the browser without checking ownership.

• **Webhook security:** verify WhatsApp and payment webhook signatures/tokens as applicable; reject forged events.

• **Booking correctness:** use transactions, conflict checks and idempotency keys for booking creation and payment webhooks.

• **Privacy:** collect only the customer details needed to fulfil the booking; define retention and deletion practices.

• **Operations:** log booking state transitions, payment events and notification failures; back up the database.

15. Suggested implementation sequence

Phase	Deliverable	Acceptance check
1. Foundation	Owner registration/login, venue creation, database and basic dashboard.	An owner can create and edit one venue.
2. Pricing & calendar	Operating hours, hourly/half-hourly rates, availability and manual blocks.	A blocked slot never appears as available to customers.
3. Customer booking	Public booking page, slot query, temporary hold and booking record.	Two simultaneous attempts cannot reserve the same slot.
4. WhatsApp	Inbound webhook, venue-code lookup and booking-link reply.	A valid code returns the correct venue link; invalid code gets a useful response.
5. Payment	UPI checkout/instructions, provider webhook or manual verification.	Only verified payment or explicit owner approval can confirm a booking.
6. Notifications & pilot	Customer WhatsApp updates, owner dashboard updates and pilot testing.	Test end-to-end with 2–3 real venues and real booking scenarios.

16. MVP acceptance checklist

■ Owner can register, sign in and see subscription status.

■ Owner can create a venue and receive a unique code and WhatsApp link.

■ Owner can set operating hours and hourly/half-hourly pricing.

■ Customer can use the WhatsApp flow and open the correct venue booking page.

■ Customer sees only genuinely available slots.

■ A temporary hold prevents another customer from selecting the same slot.

■ Owner can block and unblock eligible slots for phone/offline bookings.

■ Payment is marked paid only after provider verification or explicit owner verification.

■ Owner can approve or reject a booking that requires manual review.

■ Customer receives the correct WhatsApp status message.

■ Owner dashboard reflects the latest booking and slot status.

■ Expired holds and rejected bookings release availability correctly.

17. Decisions to confirm before development

These are the few product decisions that should be agreed before coding the booking engine:

• **Venue model:** Is one owner allowed one venue only in the first release, or multiple venues under one account?

• **Duration model:** Are customers choosing fixed 30-minute/60-minute blocks, or can they choose any start/end time in increments?

• **Payment rule:** Is full payment required to request a booking, or can the owner approve a booking before payment?

• **Hold expiry:** How long should a slot remain held while the customer pays? Ten minutes is a reasonable starting default, but confirm it.

• **Cancellation/refund:** For the MVP, decide whether cancellations are disabled, owner-managed, or supported with a simple rule.

• **Subscription enforcement:** Decide what happens to an owner's booking page and existing bookings when a subscription expires.

18. Final recommendation

Build the smallest reliable version

One owner web app + one customer booking page + one modular backend + one PostgreSQL database + official WhatsApp integration + UPI payment verification/fallback.

Prioritize slot correctness, payment-state correctness and clear customer messages. These are the parts that protect venue owners from lost time, double bookings and payment disputes.

This blueprint is a product and engineering starting point, not a final security, legal, payment-provider or WhatsApp policy review. Confirm provider capabilities, commercial terms and current platform requirements before launch.